Plantasya Mobile Application

Comprehensive IT Systems Engineering & Maintenance Manual

CHAPTER 1: SYSTEM OVERVIEW & ARCHITECTURE

1.1 Executive Summary

Plantasya is an enterprise-grade, offline-first Android application developed to facilitate machine-learning plant identification, personalized care scheduling, interactive 3D plant viewing, and library catalog management. The system operates fully offline without relying on external REST APIs, leveraging on-device TensorFlow Lite inference, Google Filament C++ 3D rendering, and local Room SQLite storage.

1.2 Full Technology Stack Specifications

Component Layer	Technology / Framework	Version / Config
Runtime Environment	Android SDK / ART Runtime	Min API 24 (7.0), Target API 35 (15)
Programming Language	Kotlin + Coroutines & Flow	Kotlin 1.9+, Coroutines 1.8+
UI Architecture	ViewBinding + Fragments + ViewPager2	AndroidX AppCompat 1.7.0
Database Engine	Room ORM (SQLite Engine)	Room 2.6.1 (Schema v12)
Machine Learning	TensorFlow Lite + CameraX API	TFLite 2.16.1, Support 0.4.4
3D Rendering Engine	Google Filament Engine	Filament Android 1.49.2
Scheduler & Alarms	AlarmManager + BroadcastReceiver	RTC_WAKEUP Exact Alarms

CHAPTER 2: DATABASE ARCHITECTURE & MAINTENANCE (ROOM ORM)

2.1 Database Schema & Entity Relationships

The local database plantasya_db (Version 12) consists of 6 entities managed via Room ORM. Foreign key constraints enforce relational integrity between users, plants, and scheduled tasks.

Entity Table	Primary Key	Foreign Keys & Constraints	Key Columns
users	user_id (Auto)	None	username, password (SHA-256), role, profile_pic (BLOB)
library_plant_tbl	id_lib (Auto)	None	plant_name, scientific_name, plant_pic (BLOB), cover_photo (BLOB), light_pt, water_pt, toxicity_plt
owned_plants	id_owned (Auto)	FK: user_id → users.user_id (CASCADE) FK: library_id → library_plant_tbl.id_lib (SET_NULL)	user_id, library_id, plant_name, plant_pic (BLOB)
tasks_tbl	id_task (Auto)	FK: plant_id → owned_plants.id_owned (CASCADE)	plant_id, task_name, task_frequency, task_done, next_reminder_time
history_tbl	history_id (Auto)	None	user_id, plant_name, is_owned, date_scanned, plant_pic (BLOB)
user_session_records	session_id (Auto)	None	user_id, username, login_time, logout_time, is_active

2.2 Automated Seeding & Self-Healing Protocol

On initial launch or database instantiation, AppDatabase.DatabaseCallback.onOpen() triggers background seeding via populateLibraryIfEmpty(). If library_plant_tbl is empty, all 25 default plants with JPEG BLOB photos are inserted. If plants exist but possess NULL photo blobs from previous schema versions, updatePlantPic() automatically re-syncs BLOB binary data without wiping user data.

2.3 Procedure: Schema Migration Escalation (Version 12 → 13+)

1. Open AppDatabase.kt and increment the version integer in @Database(version = 13).
2. Define a migration object or allow fallback: .addMigrations(MIGRATION_12_13).
3. If non-destructive migration fails at runtime, fallbackToDestructiveMigration() will rebuild tables cleanly and re-populate catalog data.

CHAPTER 3: MACHINE LEARNING & CAMERAX ENGINE MAINTENANCE

3.1 TensorFlow Lite Engine Architecture

Plant recognition is driven by ModelUnquant.tflite (Teachable Machine / MobileNet Float32 architecture). Images captured by CameraX in ScanActivity.kt are pre-processed into 224x224 RGB floating-point tensors normalized to [-1.0, 1.0].

Index	Plant Label	Index	Plant Label	Index	Plant Label
0	Aglaonema	9	Dracaena	18	Monstera Deliciosa
1	Pothos	10	Cast Iron Plant	19	Poinsettia
2	Calathea	11	Chrysanthemum	20	Polka Dot Plant
3	Money Tree	12	Ctenathe	21	Pony Tail Palm
4	Orchid	13	English Ivy	22	Prayer Plant
5	Parlor Palm	14	Dumb Cane	23	Rubber Plant
6	ZZ Plant	15	Hyacinth	24	Schefflera
7	Peace Lily	16	Jade Plant	-	-
8	Snake Plant	17	Kalanchoe	-	-

3.2 Procedure: Updating or Replacing the ML Model

1. Train new MobileNet Float32 model using 224x224 input dimensions.
2. Overwrite ModelUnquant.tflite in app/src/main/ml/.
3. If label classes change, update labels array in ScanActivity.kt to match index order.
4. Confidence threshold is set to 80% (confidenceThreshold = 0.80f). Adjust if model sensitivity requires re-calibration.

CHAPTER 4: 3D GRAPHICS & FILAMENT RENDERING MODULE

4.1 Google Filament Engine & Studio Omni-Lighting

Interactive 3D viewing is powered by Filament 3D in ThreeDPlantView.kt. A 10-point directional studio lighting setup surrounds the model to eliminate dark spots regardless of rotation. Models are automatically mapped from plant names via ThreeDPlantView.getModelFileName().

Plant Name	3D Asset File	Mapping Condition Rule
Ctenanthe	cthenathe.glb	cleanName.contains("ctenanthe") || contains("cthenathe")
Monstera Deliciosa	monstera.glb	cleanName.contains("monstera")
Poinsettia	poinsettias.glb	cleanName.contains("poinsettia")
Polka Dot Plant	polkadot_plant.glb	cleanName.contains("polka") || contains("polkadot")

Pony Tail Palm	ponytail_palm.glb	cleanName.contains("pony") || contains("ponytail")
Standard Plants (19)	[name_with_underscores].glb	cleanName.replace(" ", "_") + ".glb"

4.2 Native Memory Lifecycle Hygiene

Filament allocates C++ native memory buffers. To avoid native memory crashes or frame lag:
1. Choreographer.getInstance().removeFrameCallback(this) must execute on Activity onPause() / onDestroy().
2. All created light entities (lightEntities list) and skybox objects must be destroyed using engine.destroyEntity(light) inside onDestroy().

CHAPTER 5: CARE TASK SCHEDULING & ALARM ENGINE

5.1 Care Task Engine & Rules Matrix

Plant care tasks are automatically synced when a plant is added to dashboard via TaskSyncManager.syncTasksForPlant(). Exact RTC wakeup alarms are scheduled via AlarmManager in TaskScheduler.kt.

Rule Concept	Threshold / Duration	System Behavior & Enforcement
Checkbox Locking	Until nextReminderTime	Checkbox is disabled (isEnabled = false) while currentTime < nextReminderTime to prevent premature completion.
24-Hour Overdue Grace	24 Hours (86,400,000 ms)	Tasks are flagged Overdue ONLY when 24 hours have elapsed past nextReminderTime without action.
Urgent Overdue Alarm	Post 24-Hour Overdue	Fires high-priority notification with ic_alarm_alert, red accent, sound, and vibration.
Auto-Reset Cycle	On nextReminderTime arrival	When schedule time arrives, system resets taskDone = false, unlocking checkbox for the new cycle.
Task List Re-Sorting	Real-time / ViewObserver	Unchecked/Overdue tasks sort at top (soonest due first); completed future tasks shift to bottom.

5.2 Multi-Level Dialog Architecture

1. **Overdue Tasks Dialog (Red):** Displays 24h+ overdue tasks with quick completion buttons and top-right close 'X' button.
2. **Daily Missed Tasks Dialog (Amber):** Displays soft warning for tasks due today that remain uncompleted.
3. **Weekly Completed Dialog (Green):** Displays celebration smiley face '■ Tasks Completed!' when every task across all plants is accomplished.

CHAPTER 6: SECURITY, AUTHENTICATION & SESSIONS

6.1 Authentication & Admin Credentials

Passwords are hashed using SHA-256 inside AppDatabase.kt. The default administrator account is bootstrapped as follows:
• **Username:** admin
• **Password:** adminplantasya
• **SHA-256 Digest:** 0507a67137f814981883582496a84f3c78a9c79234b358c568f44d1808620863

6.2 Session Management

Session persistence is managed by SessionManager.kt using SharedPreferences (user_session.xml). Sessions store user_id, username, and is_logged_in status. Calling clearSession() forces user redirection to LandPage_Activity.

CHAPTER 7: OPERATIONAL MAINTENANCE PLAYBOOKS

7.1 Preventive Maintenance Routine

- **Weekly Log Audit:** Monitor Logcat for unhandled CameraCaptureException or Filament C++ memory warnings.
- **Monthly Database Maintenance:** Execute SQLite VACUUM via ADB or debug migration to optimize Room indexing.
- **Exact Alarm Audit:** Ensure device OS battery optimization settings do not suppress AlarmManager exact alarm intents.

7.2 Corrective Maintenance & Diagnostic Matrix

Observed Failure	Root Cause Analysis	Step-by-Step Resolution Procedure
Scan returns 'Unknown'	Image confidence below 80% threshold.	1. Check confidenceThreshold in ScanActivity.kt. 2. Verify lighting and 224x224 image scaling.
3D Model fails to load	Missing mapping in getModelFileName().	1. Add plant mapping rule to ThreeDPlantView.getModelFileName(). 2. Ensure .glb file exists in assets/models/.
Notifications not firing	Exact alarm permission or boot receiver issue.	1. Verify SCHEDULE_EXACT_ALARM in Android Settings. 2. Test receiver via ADB boot broadcast command.

7.3 Emergency Incident Protocols & ADB Reference Commands

■ EMERGENCY ADB COMMAND REFERENCE PLAYBOOK

1. Force Reschedule All Alarms (Alarm Deadlock Recovery):
adb shell am broadcast -a android.intent.action.BOOT_COMPLETED -n com.example.plantasya_mobileapp/.NotificationReceiver

2. Complete Application State Clear (Hard Database Reset):
adb shell pm clear com.example.plantasya_mobileapp

3. Stream Live Runtime Logs (Filament, CameraX, Room):
adb logcat -s "Plantasya" "ModelViewer" "Filament" "Room"

4. Inspect Local SQLite Tables via Shell:
adb shell run-as com.example.plantasya_mobileapp sqlite3 databases/plantasya_db "SELECT id_lib, plant_name FROM library_plant_tbl;"